

W7D1 – Esercitazione Python AI

Studente: Riccardo Luzzi

Corso: Cyber Security

Sistema utilizzato: Kali Linux

Ambiente di sviluppo: Visual Studio Code

Linguaggio: Python

Introduzione

In questa esercitazione ho configurato un ambiente Python dedicato e ho utilizzato le API di Google Gemini da uno script eseguito su Kali Linux.

Ho iniziato con una chiamata semplice al modello e successivamente ho provato alcune configurazioni, tra cui temperatura, limite dei token, System Prompt e Safety Settings. Nella parte finale ho lavorato su uno script fornito nella traccia che non funzionava subito e ho individuato gli errori fino a renderlo eseguibile.

Per tenere separati i vari passaggi ho utilizzato file Python diversi, in modo da poter verificare ogni parte senza modificare gli esercizi già funzionanti.

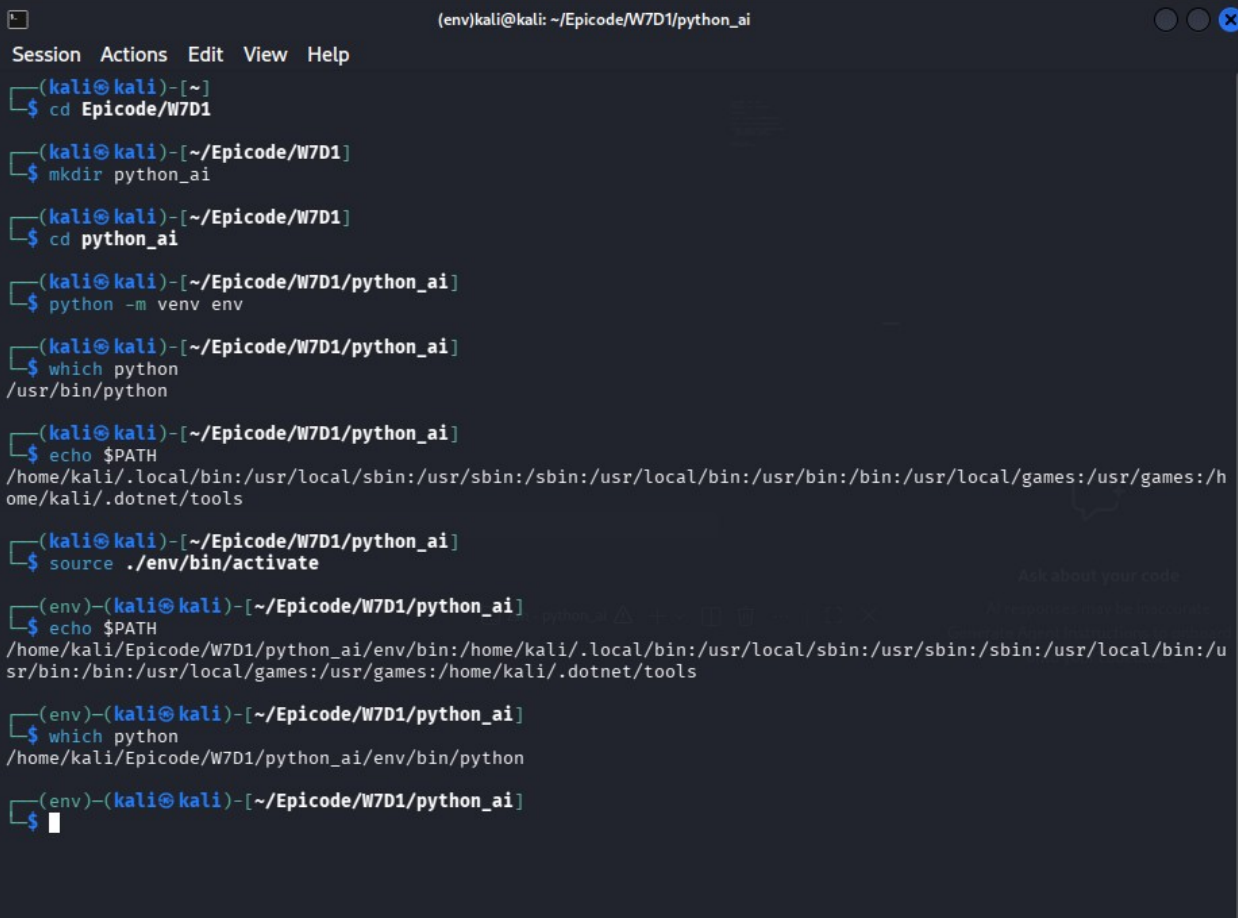
1. Preparazione dell'ambiente di lavoro

Per svolgere l'esercitazione ho creato la cartella `python_ai` all'interno di `W7D1`. Successivamente ho creato un ambiente virtuale chiamato `env`, così da installare le librerie necessarie solo per questo progetto.

- **`python -m venv env`**
- **`source ./env/bin/activate`**

Dopo l'attivazione, nel terminale compare `(env)` prima del nome utente. Ho anche verificato con `which python` che l'interprete utilizzato fosse quello presente nella cartella dell'ambiente virtuale.

Figura 1 – Creazione e attivazione dell'ambiente virtuale



```
(env)kali@kali: ~/Epicode/W7D1/python_ai
Session Actions Edit View Help
(kali@kali)-[~]
$ cd Epicode/W7D1
(kali@kali)-[~/Epicode/W7D1]
$ mkdir python_ai
(kali@kali)-[~/Epicode/W7D1]
$ cd python_ai
(kali@kali)-[~/Epicode/W7D1/python_ai]
$ python -m venv env
(kali@kali)-[~/Epicode/W7D1/python_ai]
$ which python
/usr/bin/python
(kali@kali)-[~/Epicode/W7D1/python_ai]
$ echo $PATH
/home/kali/.local/bin:/usr/local/sbin:/usr/sbin:/sbin:/usr/local/bin:/usr/bin:/bin:/usr/local/games:/usr/games:/home/kali/.dotnet/tools
(kali@kali)-[~/Epicode/W7D1/python_ai]
$ source ./env/bin/activate
(env)-(kali@kali)-[~/Epicode/W7D1/python_ai]
$ echo $PATH
/home/kali/Epicode/W7D1/python_ai/env/bin:/home/kali/.local/bin:/usr/local/sbin:/usr/sbin:/sbin:/usr/local/bin:/usr/bin:/bin:/usr/local/games:/usr/games:/home/kali/.dotnet/tools
(env)-(kali@kali)-[~/Epicode/W7D1/python_ai]
$ which python
/home/kali/Epicode/W7D1/python_ai/env/bin/python
(env)-(kali@kali)-[~/Epicode/W7D1/python_ai]
$
```

1.1 Installazione delle librerie

Con l'ambiente attivo ho installato le librerie richieste per la prova con Gemini. Oltre a google-genai ho installato pandas e numpy, come indicato nel materiale della lezione.

- **pip install google-genai pandas numpy**

Una volta terminata l'installazione ho utilizzato pip freeze per controllare i pacchetti presenti e per creare il file requirements.txt. Questo file permette di mantenere un elenco delle dipendenze e delle relative versioni utilizzate nel progetto.

- **pip freeze >> requirements.txt**

Figura 2 – Librerie installate e creazione del file requirements.txt

```
(env)-(kali㉿kali) - [~/Epicode/W7D1/python_ai]
$ pip freeze
annotated-types==0.8.0
anyio==4.14.2
certifi==2026.7.22
cffi==2.1.1
charset-normalizer==3.5.1
cryptography==50.0.0
distro==1.9.0
google-auth==2.57.0
google-genai==2.19.0
h11==0.16.0
httpcore==1.0.9
httpx==0.28.1
idna==3.19
numpy==2.5.2
pandas==3.0.5
pyasn1==0.6.4
pyasn1_modules==0.4.2
pycparser==3.0
pydantic==2.13.4
pydantic_core==2.46.4
python-dateutil==2.9.0.post0
requests==2.34.2
six==1.17.0
sniffio==1.3.1
tenacity==9.1.4
typing-inspection==0.4.4
typing_extensions==4.16.0
urllib3==2.7.0
websockets==16.1.1

(env)-(kali㉿kali) - [~/Epicode/W7D1/python_ai]
$ pip freeze >> requirements.txt

(env)-(kali㉿kali) - [~/Epicode/W7D1/python_ai]
$ ls
env requirements.txt
```

1.2 Configurazione della API Key

La chiave di Google Gemini è stata salvata nel file .env con il nome GEMINI_API_KEY. Nel codice ho quindi utilizzato python-dotenv per caricare il file senza scrivere direttamente la chiave all'interno degli script.

- **from dotenv import load_dotenv**
- **load_dotenv()**
- **api_key = os.environ["GEMINI_API_KEY"]**

Ho evitato di inserire la chiave negli screenshot e nel codice della relazione, perché deve rimanere privata.

2. Prima chiamata a Gemini

2.1 File hello_gemini.py

Nel primo file ho verificato il collegamento con Gemini effettuando una richiesta molto semplice. Il client viene creato passando la chiave API letta dal file .env e successivamente viene utilizzato il metodo `generate_content()`.

- **`client = genai.Client(api_key=api_key)`**

Questa riga crea il client utilizzato per comunicare con il servizio Gemini.

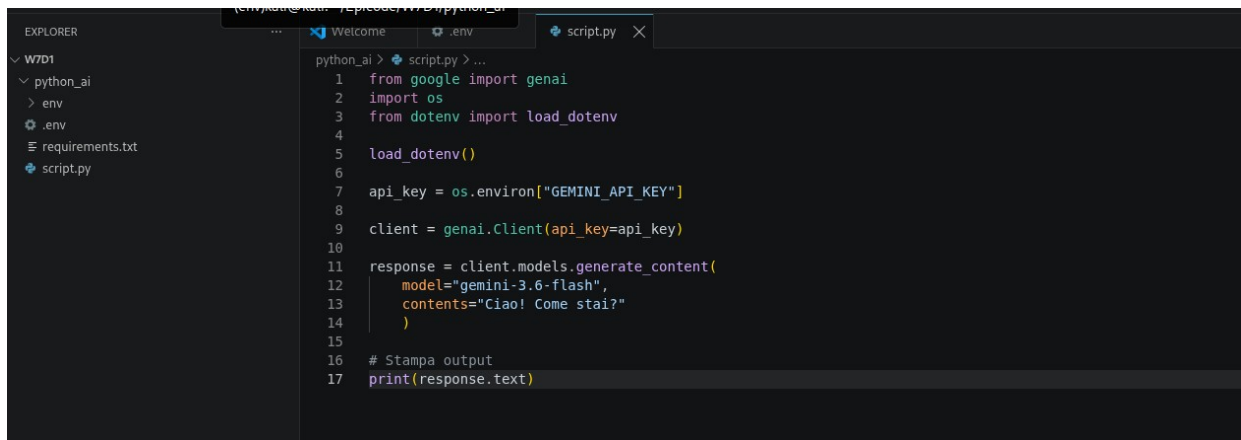
- **`response = client.models.generate_content(...)`**

Con questa chiamata viene scelto il modello e viene inviato il testo della richiesta.

- **`print(response.text)`**

Infine viene stampato nel terminale il testo restituito dal modello.

Figura 3 – Codice e prima risposta ottenuta con Gemini



```
python_ai > script.py > ...
1 from google import genai
2 import os
3 from dotenv import load_dotenv
4
5 load_dotenv()
6
7 api_key = os.environ["GEMINI_API_KEY"]
8
9 client = genai.Client(api_key=api_key)
10
11 response = client.models.generate_content(
12     model="gemini-3.6-flash",
13     contents="Ciao! Come stai?"
14 )
15
16 # Stampa output
17 print(response.text)
```

L'esecuzione ha restituito correttamente una risposta, quindi sia l'installazione della libreria sia la lettura della chiave API risultavano funzionanti.

3. Parametri utili

3.1 File parameters.py

Per il secondo test ho creato il file parameters.py. In questo caso ho importato anche types, necessario per utilizzare GenerateContentConfig e passare alcune impostazioni alla richiesta.

- **temperature=0.2**

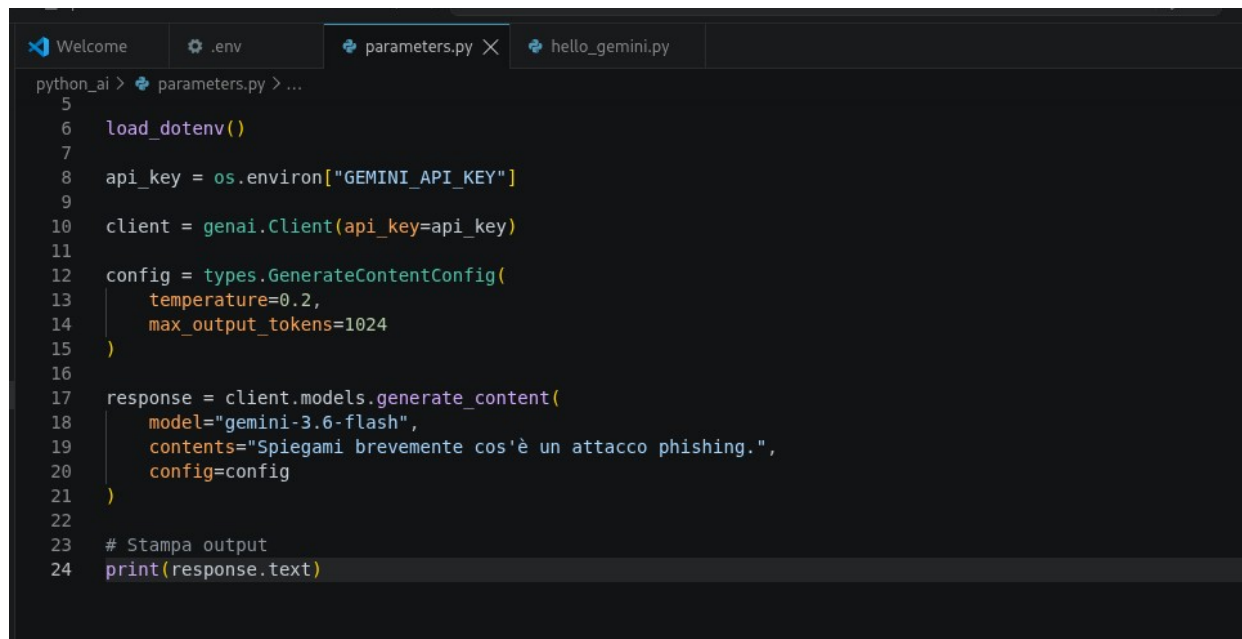
La temperatura è stata impostata a un valore basso per ottenere una risposta più precisa e poco casuale.

- **max_output_tokens=1024**

Questo parametro imposta un limite massimo alla quantità di testo che il modello può generare nella risposta.

La configurazione viene salvata nella variabile config e poi passata a generate_content tramite config=config.

Figura 4 – Codice del file parameters.py



```
python_ai > parameters.py > ...
5
6  load_dotenv()
7
8  api_key = os.environ["GEMINI_API_KEY"]
9
10 client = genai.Client(api_key=api_key)
11
12 config = types.GenerateContentConfig(
13     temperature=0.2,
14     max_output_tokens=1024
15 )
16
17 response = client.models.generate_content(
18     model="gemini-3.6-flash",
19     contents="Spiegami brevemente cos'è un attacco phishing.",
20     config=config
21 )
22
23 # Stampa output
24 print(response.text)
```

3.2 Esecuzione e verifica

Per verificare il funzionamento ho chiesto a Gemini di spiegare brevemente che cos'è un attacco phishing. Il programma ha restituito correttamente la risposta utilizzando i parametri impostati nella configurazione.

Figura 5 – Risultato dell'esecuzione di parameters.py

```
(env)-(kali@kali) - [~/Epicode/W7D1/python_ai]
$ python parameters.py
Direct use of automatic function calling (AFC) in Models.generate_content is not recommended. Instead, we recommend to use AFC in Chat.send_message. Similarly, direct use of AFC in Models.generate_content_stream is not recommended. Instead, we recommend to use AFC in Chat.send_message_stream.
Un **attacco phishing** è una truffa informatica progettata per ingannare le persone e spingerle a condividere informazioni personali e riservate, come password, numeri di carta di credito o dati bancari.

Ecco come funziona in sintesi:

1. **L'inganno:** Il truffatore invia un messaggio (via email, SMS o chat) **spacciandosi per un'entità affidabile**, come una banca, un'azienda nota (es. Amazon, Poste) o un ente governativo.
2. **L'esca:** Il messaggio crea quasi sempre un senso di urgenza o paura (es. **"Conto bloccato!"**, **"Paga
```

4. System Prompt

4.1 File system_prompt.py

Nel file `system_prompt.py` ho utilizzato `system_instruction` per definire in anticipo il comportamento dell'assistente. Ho scelto un testo semplice, indicando che l'AI deve comportarsi come un assistente amichevole per studenti di programmazione, rispondere in modo chiaro e usare esempi pratici quando possibile.

- `system_instruction=""" ... """`

Le triple virgolette permettono di scrivere l'istruzione su più righe, mantenendola più leggibile nel codice.

Come domanda ho utilizzato "Cos'è una variabile?". La risposta ottenuta ha usato un linguaggio semplice e diversi esempi, quindi il comportamento impostato nel System Prompt è stato applicato.

Figura 6 – Codice del System Prompt utilizzato

```
python_ai > system_prompt.py > ...
1  from google import genai
2  from google.genai import types
3  import os
4  from dotenv import load_dotenv
5
6  load_dotenv()
7
8  api_key = os.environ["GEMINI_API_KEY"]
9
10 client = genai.Client(api_key=api_key)
11
12 config = types.GenerateContentConfig(
13     system_instruction="""
14     Sei un assistente amichevole per studenti di programmazione.
15     Rispondi in modo semplice e chiaro.
16     Usa esempi pratici quando possibile.
17     """
18 )
19
20 response = client.models.generate_content(
21     model="gemini-3.6-flash",
22     contents="Cos'è una variabile?",
23     config=config
24 )
25
26 # Stampa output
27 print(response.text)
```

Durante l'esecuzione la risposta ha paragonato la variabile a una scatola con un'etichetta e ha mostrato anche un piccolo esempio in Python. Questo risultato mi ha permesso di verificare in modo pratico la differenza tra il contenuto della domanda e le istruzioni date al modello.

5. Safety Settings

5.1 File safety_settings.py

Nel quarto file ho provato le impostazioni di sicurezza mostrate durante la lezione. Ho creato una lista di SafetySetting e ho impostato BLOCK_NONE per le categorie indicate nell'esempio, utilizzando gli enum messi a disposizione dalla versione della libreria installata.

Le categorie utilizzate nel test sono contenuti pericolosi, hate speech, harassment e contenuti sessualmente espliciti. Le impostazioni sono state poi inserite dentro GenerateContentConfig insieme a temperature=0.2.

Figura 7 – Configurazione delle Safety Settings

```
python_ai > safety_settings.py > ...
1  from google import genai
2  from google.genai import types
3  import os
4  from dotenv import load_dotenv
5
6  load_dotenv()
7
8  api_key = os.environ["GEMINI_API_KEY"]
9
10 client = genai.Client(api_key=api_key)
11
12 safety_settings = [
13     types.SafetySetting(
14         category=types.HarmCategory.HARM_CATEGORY_DANGEROUS_CONTENT,
15         threshold=types.HarmBlockThreshold.BLOCK_NONE
16     ),
17     types.SafetySetting(
18         category=types.HarmCategory.HARM_CATEGORY_HATE_SPEECH,
19         threshold=types.HarmBlockThreshold.BLOCK_NONE
20     ),
21     types.SafetySetting(
22         category=types.HarmCategory.HARM_CATEGORY_HARASSMENT,
23         threshold=types.HarmBlockThreshold.BLOCK_NONE
24     ),
25     types.SafetySetting(
26         category=types.HarmCategory.HARM_CATEGORY_SEXUALLY_EXPLICIT,
27         threshold=types.HarmBlockThreshold.BLOCK_NONE
28     )
29 ]
30
31 config = types.GenerateContentConfig(
32     safety_settings=safety_settings,
33     temperature=0.2
34 )
35
36 response = client.models.generate_content(
37     model="gemini-3.6-flash",
38     contents="Spiegami brevemente cos'è il social engineering.",
39     config=config
40 )
41
42 print(response.text)
```

5.2 Verifica del funzionamento

Per il test ho chiesto una spiegazione del social engineering. Lo script ha funzionato e Gemini ha restituito una risposta descrivendo il concetto e alcuni esempi, tra cui phishing, vishing, smishing e pretexting.

Nel terminale compare anche un messaggio relativo all'Automatic Function Calling (AFC). Si tratta di un avviso della libreria e non di un errore, infatti l'esecuzione continua normalmente e viene stampata la risposta.

Figura 8 – Esecuzione del file safety_settings.py

```
(env)-(kali@kali)-[~/Epicode/W7D1/python_ai]
$ python safety_settings.py
Direct use of automatic function calling (AFC) in Models.generate_content is not recommended. Instead, we recommend to use AFC in Chat.send_message. Similarly, direct use of AFC in Models.generate_content_stream is not recommended. Instead, we recommend to use AFC in Chat.send_message_stream.
Il social engineering (in italiano ingegneria sociale) è una tecnica di manipolazione psicologica usata dai cybercriminali per spingere le persone a compiere azioni imprudenti, come condividere informazioni confidenziali, accedere a link malevoli o trasferire denaro.

Invece di violare un computer sfruttando difetti del software, il social engineering hackera l'essere umano. Sfrutta le debolezze psicologiche naturali dell'uomo, come la fiducia, la paura, la curiosità, il rispetto per l'autorità o il senso di urgenza.

### Esempi più comuni:

* Phishing: Email o messaggi falsi che sembrano provenire da fonti affidabili (banche, Poste, colleghi) per rubare password o dati della carta di credito.
* Vishing e Smishing: La stessa truffa del phishing, ma realizzata tramite chiamate vocali (Vishing) o SMS (Smishing).
* Pretexting (Pretesto): Il malintenzionato inventa una storia credibile (es. fingendosi un tecnico dell'assistenza informatica) per farsi dare le credenziali di accesso.
* Baiting (Esca): Offrire qualcosa di allettante (es. un film gratis da scaricare o una chiavetta USB "smarrita" in un parcheggio) che contiene in realtà un virus.

### In sintesi:
La regola d'oro contro il social engineering è lo scetticismo: verificare sempre l'identità di chi chiede informazioni riservate e non agire mai d'istinto.

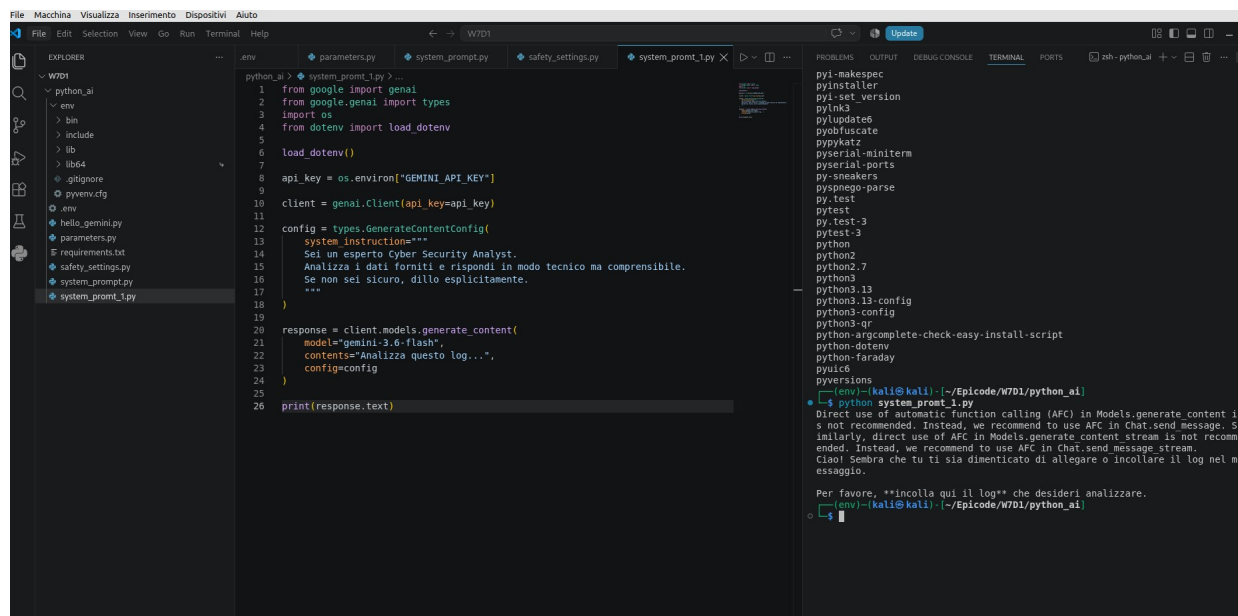
(env)-(kali@kali)-[~/Epicode/W7D1/python_ai]
$
```

6. System Prompt orientato alla Cyber Security

Prima dell'esercizio finale ho effettuato un secondo test del System Prompt, questa volta impostando il ruolo di Cyber Security Analyst. L'istruzione chiede di analizzare i dati in modo tecnico ma comprensibile e di dichiarare quando non si è sicuri della risposta.

Come contenuto ho inserito la richiesta “Analizza questo log...”. Il modello ha risposto chiedendo di incollare il log da analizzare, quindi anche questo System Prompt è stato interpretato correttamente.

Figura 9 – Secondo System Prompt e risultato nel terminale



7. Esercizio finale: individuazione e correzione degli errori

Nell'ultima parte la traccia forniva uno script per una conversazione continua con Gemini, specificando che probabilmente non avrebbe funzionato subito a causa dei cambiamenti nelle API di Google. Il compito era quindi individuare il problema e rendere funzionante lo script.

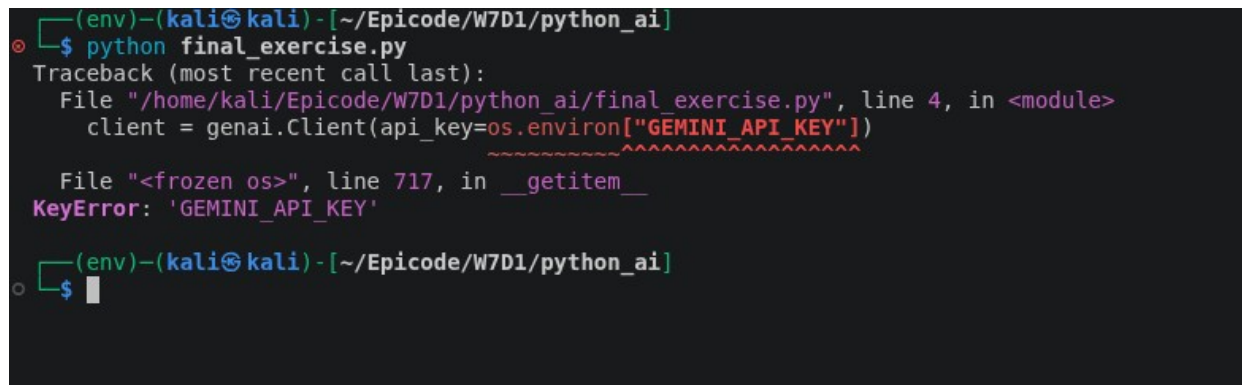
7.1 Primo errore: GEMINI_API_KEY

Ho copiato inizialmente il codice della slide e l'ho eseguito senza modifiche. Il primo errore ottenuto è stato un `KeyError` relativo a `GEMINI_API_KEY`.

Il problema era che lo script cercava la variabile con `os.environ`, ma non caricava il file `.env`. Negli esercizi precedenti questo passaggio veniva effettuato tramite `load_dotenv()`.

- `from dotenv import load_dotenv`
- `load_dotenv()`

Figura 10 – Primo errore: variabile `GEMINI_API_KEY` non caricata



```
(env)-(kali@kali)-[~/Epicode/W7D1/python_ai]
$ python final_exercise.py
Traceback (most recent call last):
  File "/home/kali/Epicode/W7D1/python_ai/final_exercise.py", line 4, in <module>
    client = genai.Client(api_key=os.environ["GEMINI_API_KEY"])
                                     ~~~~~~^~~~~~
  File "<frozen os>", line 717, in __getitem__
KeyError: 'GEMINI_API_KEY'

(env)-(kali@kali)-[~/Epicode/W7D1/python_ai]
$
```

Dopo aver aggiunto il caricamento del file `.env`, il programma è riuscito ad avviarsi e ha mostrato il prompt "Tu:", entrando correttamente nel ciclo `while`.

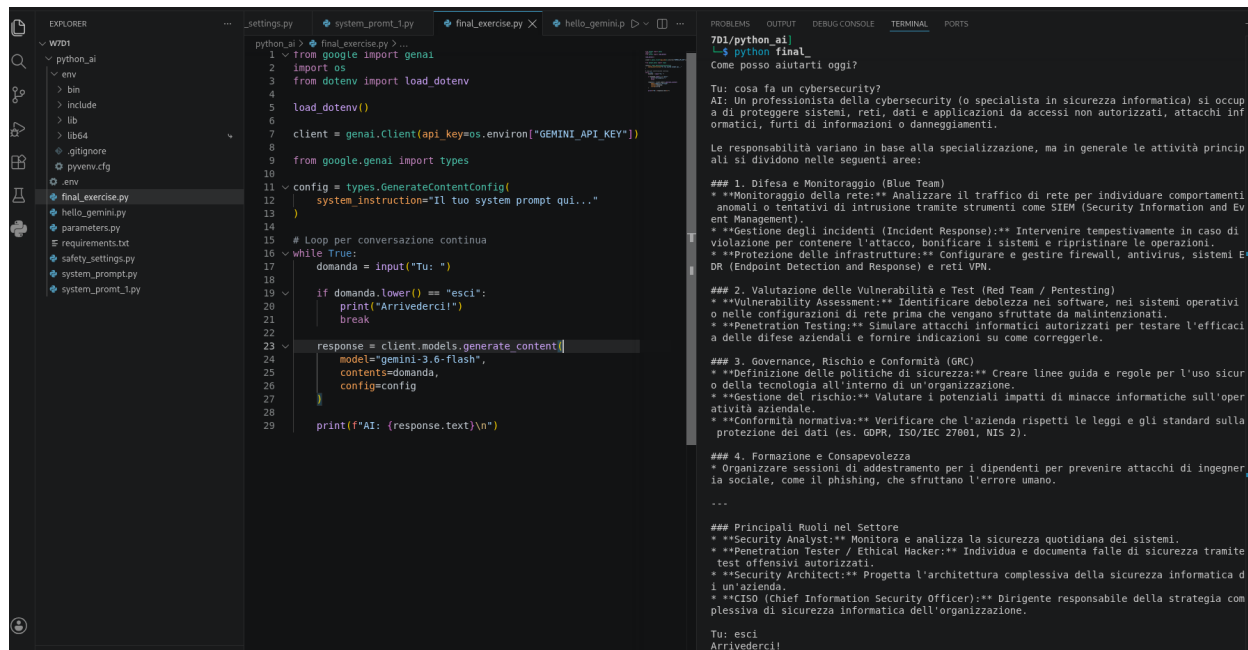
[illegible]

7.3 Verifica finale della conversazione

Dopo le due correzioni lo script ha iniziato a funzionare. Ho effettuato più domande per verificare che il ciclo while continuasse a chiedere un nuovo input dopo ogni risposta.

Ho chiesto anche “Cosa fa un cybersecurity?” e il modello ha restituito una risposta articolata sulle principali attività del settore. Infine ho scritto “esci” e il programma ha stampato “Arrivederci!”, terminando correttamente il ciclo.

Figura 12 – Script finale funzionante, conversazione continua e comando di uscita



The screenshot shows a code editor with a file explorer on the left, a Python script in the center, and a terminal window on the right. The script, named `final_esercizio.py`, imports `google.generativeai` and `os`, loads `dotenv`, and initializes a `genai` client using an API key from environment variables. It then sets up a `types.GenerateContentConfig` with a system instruction and enters a `while` loop that prompts the user and generates responses from the Gemini model. The terminal shows the execution of the script, which prints a detailed overview of cybersecurity roles and responsibilities, and finally prints "Arrivederci!" after the user enters "esci".

```
python3 final_esercizio.py
1 from google import genai
2 import os
3 from dotenv import load_dotenv
4
5 load_dotenv()
6
7 client = genai.Client(api_key=os.environ["GEMINI_API_KEY"])
8
9 from google.generativeai import types
10
11 config = types.GenerateContentConfig(
12     system_instruction="Il tuo system prompt qui..."
13 )
14
15 # Loop per conversazione continua
16 while True:
17     domanda = input("Tu: ")
18
19     if domanda.lower() == "esci":
20         print("Arrivederci!")
21         break
22
23     response = client.models.generate_content(
24         model="gemini-3.6-flash",
25         contents=domanda,
26         config=config
27     )
28
29     print(f"AI: {response.text}\n")
```

Terminal Output:

```
701/python_ai]
701/python final_
701/python final_
Come posso aiutarti oggi?

Tu: cosa fa un cybersecurity?
AI: Un professionista della cybersecurity (o specialista in sicurezza informatica) si occupa di proteggere sistemi, reti, dati e applicazioni da accessi non autorizzati, attacchi informatici, furti di informazioni o danneggiamenti.

Le responsabilità variano in base alla specializzazione, ma in generale le attività principali si dividono nelle seguenti aree:

### 1. Difesa e Monitoraggio (Blue Team)
* **Monitoraggio della rete:** Analizzare il traffico di rete per individuare comportamenti anomali o tentativi di intrusione tramite strumenti come SIEM (Security Information and Event Management).
* **Gestione degli incidenti (Incident Response):** Intervenire tempestivamente in caso di violazione per contenere l'attacco, bonificare i sistemi e ripristinare le operazioni.
* **Protezione delle infrastrutture:** Configurare e gestire firewall, antivirus, sistemi EDR (Endpoint Detection and Response) e reti VPN.

### 2. Valutazione della Vulnerabilità e Test (Red Team / Pentesting)
* **Vulnerability Assessment:** Identificare debolezze nel software, nei sistemi operativi o nelle configurazioni di rete prima che vengano sfruttate da malintenzionati.
* **Penetration Testing:** Simulare attacchi informatici autorizzati per testare l'efficacia delle difese aziendali e fornire indicazioni su come correggerle.

### 3. Governance, Rischio e Conformità (GRC)
* **Definizione delle politiche di sicurezza:** Creare linee guida e regole per l'uso sicuro della tecnologia all'interno di un'organizzazione.
* **Gestione del rischio:** Valutare i potenziali impatti di minacce informatiche sull'attività aziendale.
* **Conformità normativa:** Verificare che l'azienda rispetti le leggi e gli standard sulla protezione dei dati (es. GDPR, ISO/IEC 27001, NIS 2).

### 4. Formazione e Consapevolezza
* Organizzare sessioni di addestramento per i dipendenti per prevenire attacchi di ingegneria sociale, come il phishing, che sfruttano l'errore umano.

...

### Principali Ruoli nel Settore
* **Security Analyst:** Monitora e analizza la sicurezza quotidiana dei sistemi.
* **Penetration Tester / Ethical Hacker:** Individua e documenta falle di sicurezza tramite test offensivi autorizzati.
* **Security Architect:** Progetta l'architettura complessiva della sicurezza informatica di un'azienda.
* **CISO (Chief Information Security Officer):** Dirigente responsabile della strategia complessiva di sicurezza informatica dell'organizzazione.

Tu: esci
Arrivederci!
```

L'esercizio finale è quindi stato risolto individuando due problemi distinti: il mancato caricamento della variabile dal file `.env` e il nome del modello non disponibile nella configurazione utilizzata.

8. Conclusioni

Nel complesso sono riuscito a completare tutti i passaggi dell'esercitazione e a far funzionare gli script separatamente. La prima parte mi è servita per configurare correttamente l'ambiente virtuale e capire come leggere la chiave API senza inserirla direttamente nel codice.

Con i file successivi ho provato alcune configurazioni di Gemini, in particolare temperatura, limite dei token, System Prompt e Safety Settings. Tenere gli esercizi in file separati mi ha aiutato a capire meglio cosa cambiava da una prova all'altra e a non perdere una versione già funzionante.

La parte più utile è stata l'esercitazione finale, perché il codice non funzionava subito e ho dovuto leggere gli errori del terminale. Prima ho risolto il problema della variabile `GEMINI_API_KEY` e successivamente quello del modello `gemini-3-flash`. Dopo le correzioni il programma è riuscito a gestire più domande consecutive e a chiudersi con il comando “esci”.

Questa prova mi ha fatto vedere in modo pratico che, quando si utilizzano servizi esterni e API, un esempio trovato nelle slide o nella documentazione può richiedere modifiche in base alla versione della libreria o ai modelli disponibili.